

Algorithms & Data Structures (M)

Assessed Exercise (2025–26)

This exercise is assessed. Its weighting is 0.2 in the ADS(M) course assessment. The deadline is *Monday 26 March 2026* at 16:30.

There are two parts to the assessment. Part A can be completed straight away (you will find the fruit market example from lecture 1 useful). Part B cannot be completed until after we have studied Sets and Binary Search Trees in the lectures (approximately March 2nd).

A. This part is to test that you understand how to use Java's `Collection` classes, specifically the `Set` classes (interface and implementations) in the package `java.util`.

A Bag is a collection that is similar to a `Set`, except that allows for multiple copies of the same element. In this exercise you are going to imitate a Bag using two sets, one to record the unique elements in the bag and the other to record how many copies of each element are in the bag. You have been provided with class outlines `CountedElement.java` and `WordProcessor.java`, and three text files, `file0.txt`, `file1.txt` and `file2.txt`.

(a) Complete the class `CountedElement.java`. Do not add any additional instance variables or methods - just complete the methods as described in the comments.

(2 marks)

(b) Complete outline file `WordProcessor.java`, by following the commented instructions. When run from the command line using the following commands:

```
>javac CountedElement.java
>javac WordProcessor.java
>java WordProcessor.java "file0.txt" "file1.txt" "file2.txt"
```

The following should be output (to standard output):

```
(ant,4), (biscuit,1), (box,3), (brain,1), (camping,1),
(cow,5), (jumper,1), (kettle,3), (kilt,1), (lathe,1),
(present,1), (pub,1), (table,1), (wonder,1), (xylophone,1),
(yak,1), (yoyo,1), (zany,1), (zeal,2), (zoo,3),
```

(4 marks)

B. This exercise is about implementation of an abstract data type (ADT). Specifically it involves implementing the supplied `Bag` interface. Note that you must not alter the interface (i.e. you cannot add methods, change the type etc.)

You must implement the `Bag` interface as a class `BSTBag.java`, using a binary search tree (not an AVL tree) whose nodes have elements of type

`CountedElement<E>`. (You should assume that the working directory contains your `CountedElement<E>` implementation from part A.) You should write a test program, `WordProcessor2.java` say, to test your implementation, but you will not submit your test file. I will test your implementation using my own test file, only referring to the methods in the interface. When adding (removing) an item of type `E` to (from) the bag, you should check to see if the same element has been added previously and increment (decrement) the integer part of the stored `CountedElement` accordingly.

As well as completing your `BSTBag.java` class you must provide a pdf document “`algorithms.pdf`” containing the *algorithms* used in your `add`, `remove` and `iterator` implementations. These should be adapted from the corresponding algorithms for the BST implementation of a set given in the course notes and should *not* contain any java code. The algorithms contained in this document will contribute to the total mark awarded for the associated method. Note that the `iterator` algorithm must return elements of type `E` (not of type `CountedElement<E>`).

Marks for part B will be allocated as follows:

Instance variables, `Node` inner class, simple methods – 3 marks

`Add` – 5

`Remove` – 5

`Iterator` - 7 marks

`Contains` – 2 marks

`equals` - 2 marks

Note that classes `Stack.java` and `LinkedStack.java` have been provided for use in your `iterator` implementation. You should not rely on any other additional classes in your implementation.

Submission

By the deadline stated above, you must submit your deliverables through the ADS(M) Moodle page. (Click “Assessed exercise submission”.)

Submit all the deliverables as a single `.zip` file.

Please note the following important submission instructions. Throughout, replace `xxxxxxx` by your student number (i.e. the first part of your email address without the letter).

1. All files should be contained within a single (zipped) directory.
2. Your zip file must be named thus:

`xxxxxxxADS.zip`

3. The zipped folder (`xxxxxxxADS`) must contain two subdirectories:

`xxxxxxxExA` and `xxxxxxxExB`

4. Folder `xxxxxxxExA` must contain completed files:

CountedElement.java and WordProcessor.java

5. Folder xxxxxxxxB must contain the following:

BSTBag.java and algorithms.pdf

In addition note that:

All submission files must contain (as a comment, in the case of java files) your student id and name, and any package declaration should be removed (or commented out). Failure to comply with ALL submission instructions will lead to a deduction of (or no) marks.

Important: You must also sign the School's "declaration of originality" form.